Chunking and Deduplication

think in chunks, not files

Watch the 4-min video

In one line: split files into chunks, hash each one, and upload only the chunks the server doesn't already have; content-defined chunking (a rolling hash picks the boundaries) keeps an insert from shifting every chunk, so a new version costs only its changed chunks: that's delta sync and deduplication.

1 500 MB for a one-slide edit

You change one slide in a 500 MB presentation and the sync client re-uploads all 500 MB: ten minutes on a slow uplink for a few kilobytes of real change. Across millions of users this is mostly wasted bandwidth and storage, because most of each new version is identical to the previous one, and many files (installers, shared videos, photos forwarded around) are identical across users. Sync and backup systems stop treating files as the unit of transfer and storage; they use **chunks**.

2 Chunk, hash, ask

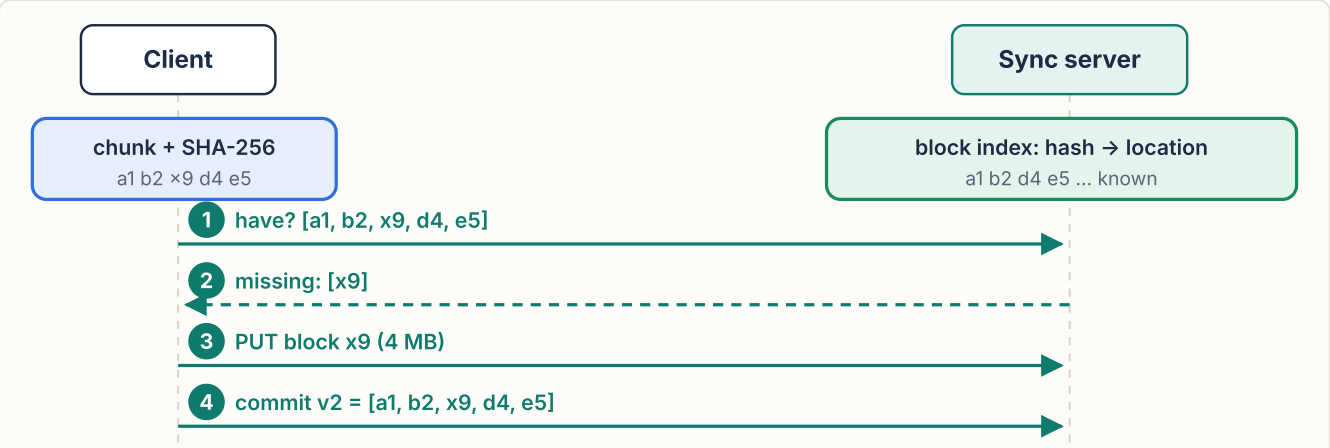


Figure 1. Block-level deduplication: the client sends hashes first, and the server says which chunks it has never seen.

1. The client splits the file into chunks and computes a strong hash (SHA-256) of each.
2. It sends the list of hashes; the server checks its block index and replies with the ones it lacks.
3. Only missing chunks are uploaded (each stored once, content-addressed, lesson 7).
4. The client commits the new version as an ordered list of hashes. Unchanged chunks, from this file, an older version or another file, are referenced again.

Downloads work the same way in reverse: the client fetches only the chunks it doesn't hold locally.

3 Fixed-size chunking and the insert problem

Fixed-size chunking cuts every N bytes (Dropbox uses 4 MB blocks). It's simple and fast, and it works well when edits overwrite bytes in place, as in databases, VM images or many media formats. But insert or delete even one byte near the start of a file and every later boundary shifts: each chunk now holds different bytes, every hash changes, and the whole file looks new.

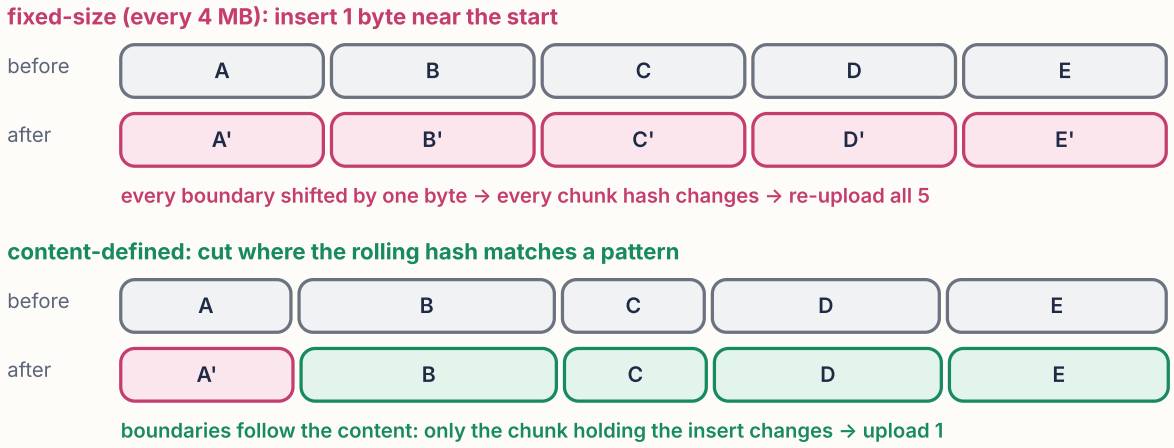
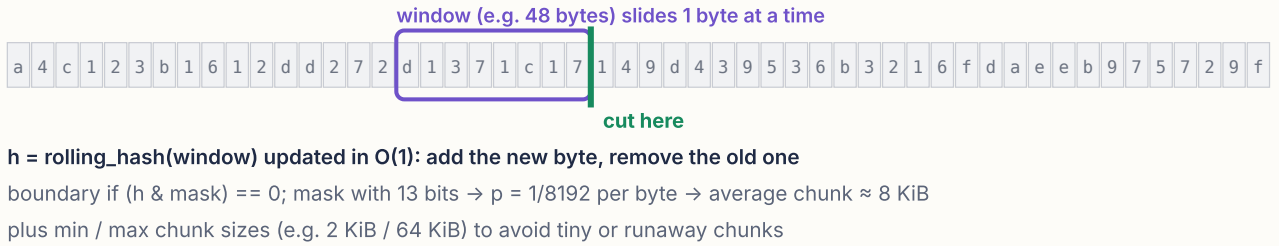


Figure 2. Fixed boundaries are positions; content-defined boundaries are patterns in the data, so they move with the data.

4 Content-defined chunking (CDC)



h = rolling_hash(window) updated in O(1): add the new byte, remove the old one

boundary if (h & mask) == 0; mask with 13 bits → p = 1/8192 per byte → average chunk ≈ 8 KiB

plus min / max chunk sizes (e.g. 2 KiB / 64 KiB) to avoid tiny or runaway chunks

Figure 3. A rolling hash over a small sliding window decides where chunks end. The same content produces the same cut points wherever it sits in the file.

Slide a small window over the data and compute a **rolling hash** of it at every byte; the hash updates in constant time as one byte enters and one leaves. Declare a boundary wherever the hash has a chosen bit pattern, e.g. its low 13 bits are zero. Because the decision depends only on the bytes in the window, an insert changes the boundaries only near the edit; after it, the same content produces the same cut points, so the following chunks keep their hashes. The expected chunk size is about 2^k bytes for a k-bit mask, bounded by minimum and maximum sizes.

System	Chunking	Typical sizes
rsync (Tridgell, 1996)	receiver's file in fixed blocks; sender's rolling weak checksum finds them at any offset, a strong hash confirms	block size scales with file size
LBFS (SOSP 2001)	CDC with Rabin fingerprints over a 48-byte window	average 8 KB, min 2 KB, max 64 KB
Dropbox	fixed-size blocks, SHA-256	4 MB
restic / Borg	CDC (Rabin / buzhash) for backups	about 1–2 MiB average (512 KiB–8 MiB)
FastCDC (USENIX ATC 2016)	CDC with a cheaper "Gear" hash and normalised sizes	configurable; several times faster than Rabin CDC

5 Delta sync

With chunking, a new version is just a new list of hashes, most of them already stored. **Delta sync** moves only the changed chunks in both directions; unchanged ones are referenced again. The same mechanism deduplicates **across** files and users: a popular installer is stored once no matter how many people have it. Two related ideas: rsync computes a delta between two specific files, without a global chunk store; and some services add **compression** per chunk and **streaming** (upload chunks while the file is still being written).

6 Trade-offs: chunk size and privacy

Chunk size. Smaller chunks find more duplicates and make edits cheaper, but cost more hashes, index entries and requests. Rough numbers for 1 PB of unique data, with ~40 bytes per index entry (32-byte hash + location): 8 KiB chunks → ~1.2×10^{11} chunks and ~5 TB of index; 4 MiB chunks → ~2.4×10^{8} chunks and ~10 GB of index. Object stores also charge per request, so tiny chunks are often packed into larger objects ("pack files").

Privacy. Cross-user dedup is a side channel: if uploading a file finishes instantly, someone else already has it. Attackers can confirm that a known file exists, or guess a secret inside a templated document by trying variants. And if the server links a user to content just because they sent its hash, a leaked hash becomes a download token (2011's "Dropship" did this with Dropbox hashes). Mitigations: dedup only within one account, dedup server-side only (client always uploads; saves storage, not bandwidth), or ask the client to prove possession (e.g. hashes of random byte ranges). End-to-end encryption with per-user keys makes identical files look different, which disables cross-user dedup unless you use *convergent encryption* (key derived from the content), which reintroduces the equality leak.

TRADE-OFFS

- Fixed-size: fast and simple, perfect for in-place edits; fails on inserts and deletes.
- CDC: robust to shifts; costs hashing CPU and gives variable chunk sizes.
- Small chunks: better dedup, more metadata and requests. Large chunks: the reverse.
- Cross-user dedup: big savings, but a privacy side channel; per-account dedup is safer.

7 Example: only the changed 4 MB blocks

WORKED EXAMPLE: A 2 GB VIDEO PROJECT, A FEW SECONDS CHANGED

The file is 2,000 MB, so with fixed 4 MB blocks it's **500 blocks**. The user re-renders a few seconds in the middle; the project format overwrites bytes in place, so the file length and all other offsets stay the same. The client re-chunks and hashes the file (SHA-256 runs at hundreds of MB/s per core, so a few seconds), sends 500 hashes (16 KB of hashes), and the server replies that it lacks 2. The client uploads **2 blocks = 8 MB** instead of 2 GB, and the server records version 2 as the new block list; version 1 keeps its list, so restoring it is free.

Had the edit *inserted* data (adding a frame to a format that isn't rewritten in place), fixed blocks after the insert would all shift and ~half the file would re-upload. With CDC only the chunks around the insert would change.

COMMON MISTAKES

- Using fixed-size chunks for data that is edited by insertion (text, archives rebuilt from scratch).
- Using a weak or non-cryptographic hash (CRC, MD5) as the chunk identity.
- Linking content to a user on hash alone, without proof of possession.
- Ignoring index and request overhead when choosing a tiny average chunk size.

8 Key terms

Chunk / block: piece of a file, identified by the hash of its bytes

Content-defined chunking: boundaries chosen by content, not position

Delta sync: transfer only the chunks that changed between versions

Block-level dedup: store and send each distinct chunk only once

Rolling hash: hash of a sliding window updated in O(1) per byte (Rabin, buzhash, Gear)

Convergent encryption: key derived from content so equal files encrypt equally

9 Interview questions

Q1. How does a file sync service avoid re-uploading unchanged data? BASIC

It splits files into chunks, hashes them, and uploads only chunks the server doesn't have; content-defined chunking with a rolling hash keeps an insert from shifting every boundary.

Q2. What is delta sync? BASIC

Sending only the changed chunks of a new version, while unchanged chunks are referenced again by hash.

Q3. Users edit large text-based CSV exports by inserting rows near the top. Fixed or CDC? SCENARIO

Content-defined chunking: inserts shift every fixed boundary, while CDC boundaries follow the content and only nearby chunks change.

Q4. Design dedup for a backup product storing many VM images. SCENARIO

CDC with ~1–8 MiB average chunks, SHA-256 identities, chunks packed into larger objects, a chunk index, and per-snapshot chunk lists with ref counting or mark-and-sweep GC.

Q5. How do you pick the average chunk size? SCENARIO

Balance dedup ratio against index size and request counts: estimate chunks = data ÷ chunk size and the memory/DB cost of the index; MB-scale for cloud sync and backups, KB-scale only with packing.

Q6. Why can cross-user deduplication leak information? TRICKY

An instant upload reveals the server already has that content, so an attacker can test whether a file (or a guessed variant of a document) exists; per-account dedup or proof of possession limits this.

Q7. Why does CDC need minimum and maximum chunk sizes? TRICKY

Without a minimum, unlucky data produces tiny chunks (index blow-up); without a maximum, data that never matches the pattern (e.g. long runs of zeros) produces huge chunks that dedup poorly.

Next: SD-O-009 · Image Processing backendroadmap.com